

FILE SYSTEM IMPLEMENTATION-
VERY SIMPLE FILE SYSTEM

Understanding File Systems: Two Key Aspects

- **Data Structures (On-Disk Organization)**

- Organize **data** and **metadata**
- Basic: Arrays of blocks (e.g., early file systems like vsfs)
- Advanced: Tree-based structures (e.g., SGI's XFS)

- **Access Methods (System Call Mapping)**

- How system calls (e.g., `open()`, `read()`, `write()`) interact with structures
- Determines:
- **Which structures are accessed** (read/write)
- **Performance** of file operations

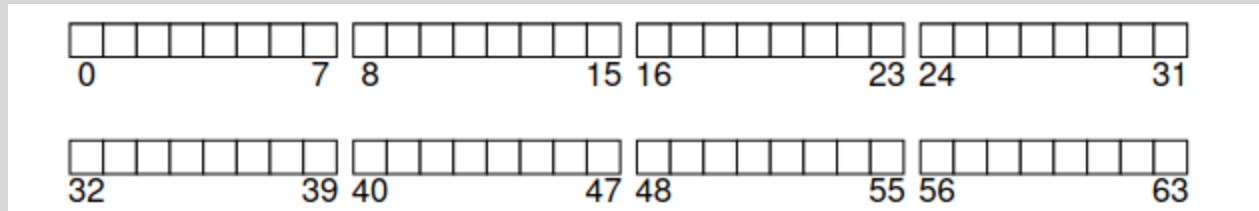
Overall Organization

- Block-Based Disk Layout

- Disk is divided into **fixed-size blocks**
- **Block size:** 4 KB (common and simple)
- Disk is viewed as a **sequence of blocks**, indexed from **0 to N-1**

Example

- Small disk with **64 blocks**
- Total disk size: **$64 \times 4 \text{ KB} = 256 \text{ KB}$**

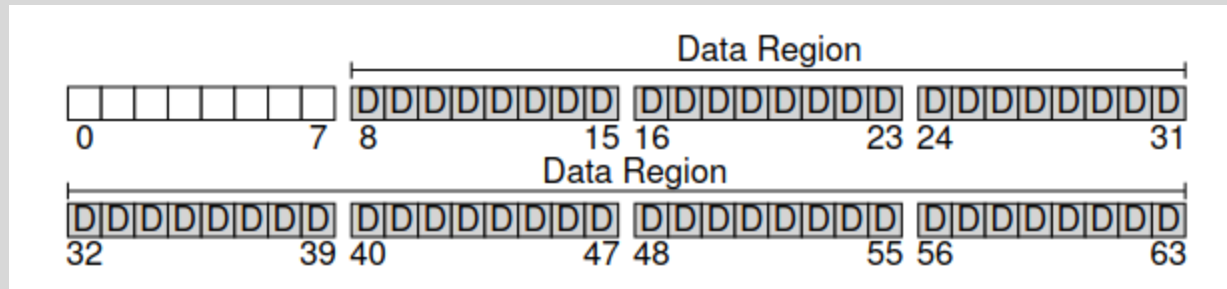


What Goes on Disk?

- What we need to store:
- **User Data** – majority of disk space (Data Region)
- **Metadata** – info about each file (e.g., size, owner, block locations)
 - **Inodes** – structure to hold per-file metadata
- **Allocation Info** – track which blocks/inodes are free
- **Superblock** – holds file system layout and key info

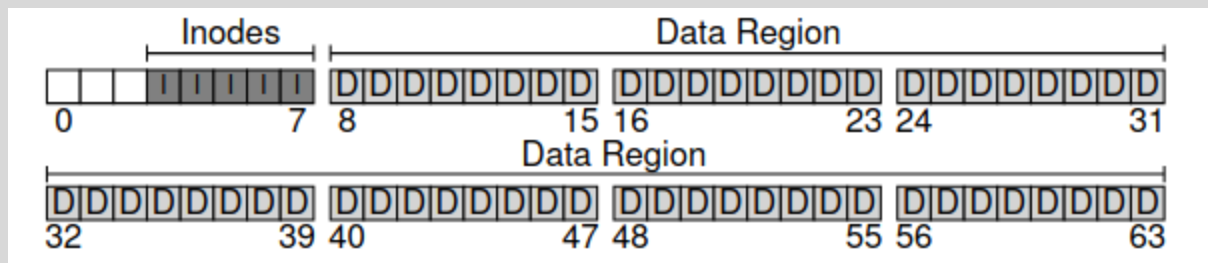
Data Region

- User Data Storage
 - Majority of disk blocks are used for user files
 - Called the **Data Region**
 - In this example: **Last 56 out of 64 blocks** are for user data
- **Block 8–63** → Data Region (D)



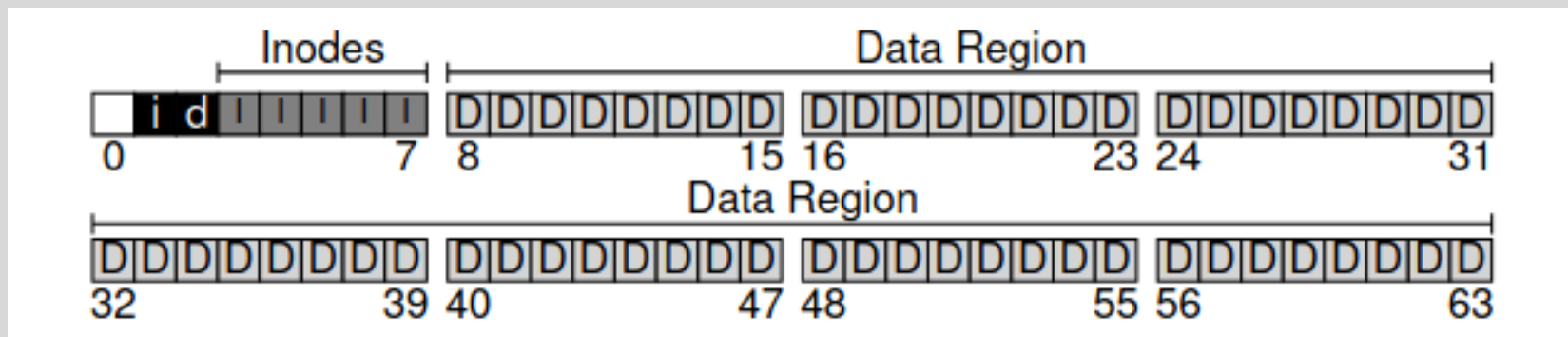
Inode and Inode Table

- Inode: Stores metadata for each file
 - Info stored: File type (e.g., regular file, directory), file size, number of allocated blocks, protection info (owner, permissions), timestamps (created, modified, accessed), and pointers to data blocks (file contents location).
 - Stored in **Inode Table**
 - In this example: **5 blocks used for inodes**
- **Block 3–7** → Inode Table (I)
- Inode Size:
- Assuming 256 bytes/inode → 16 inodes per 4KB block
→ 5 blocks × 16 = **80 inodes (max 80 files)**



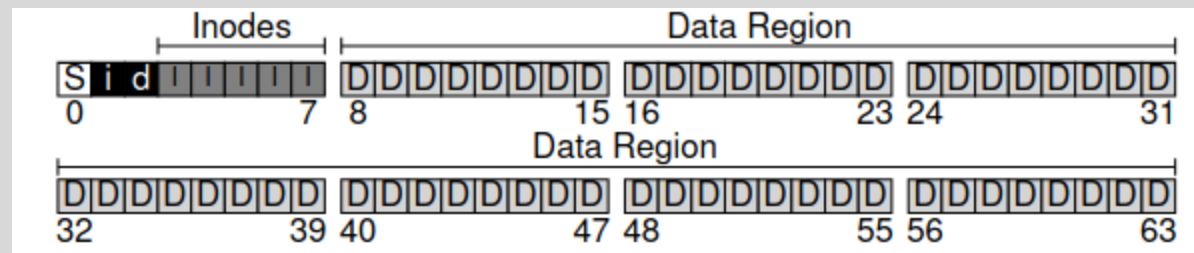
Bitmap

- Bitmaps for Tracking Free/Used Blocks- free a bit 0 / in-use- a bit 1
- **Inode Bitmap (i)** → 1 bit per inode
- **Data Bitmap (d)** → 1 bit per data block
 - **Block 2** → Inode Bitmap (i)
 - **Block 3** → Data Bitmap (d)
- Even though only 80 inodes and 56 data blocks exist, each bitmap can track up to 32,768 entries (1 bit per inode/block × 4KB × 8 bits).
- For simplicity we have considered one block per bitmap



Super block

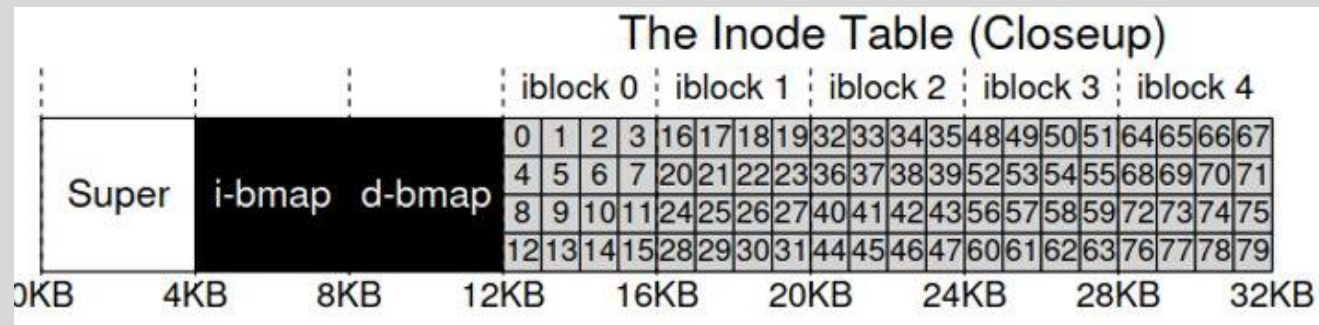
- **One block reserved** for the superblock (denoted **S**)
- Holds **critical information** about the file system, including:
- Total number of **inodes** (e.g., 80)
- Total number of **data blocks** (e.g., 56)
- Starting block of the **inode table** (e.g., block 3)
- A **magic number** that uniquely identifies the file system type (here, vsfs)



- **Inode = Index Node**
- Core on-disk structure in almost all file systems (e.g., UNIX)
- Stores **metadata** about a file (not file data itself)
- Each inode identified by an **i-number** (low-level file identifier)
- In our example, the Inode table size: **20 KB** (5 blocks × 4 KB)
- Contains **80 inodes** (256 bytes per inode)
- Inode table starts at **12 KB** on disk
- To find a particular inode:

$$\text{blk} = (\text{inumber} \times \text{sizeof}(\text{inode_t})) / \text{blockSize};$$

$$\text{sector} = ((\text{blk} \times \text{blockSize}) + \text{inodeStartAddr}) / \text{sectorSize};$$



Locating an Inode on Disk

Find the physical disk location of a specific inode (e.g., inode #32).

Step 1: Understand the inode size and inode table start

Each inode is **256 bytes** in size; Inode table starts at **12 KB** (on the disk); Disk block size: **4 KB** (4096 bytes) & Sector size: **512 bytes**

Step 2: Calculate byte offset of the inode inside the inode table

$\text{inode offset} = \text{inode number} \times \text{inode size}$

For inode #32: $32 \times 256 = 8192$

Step 3: Calculate the absolute byte address on disk

$\text{absolute address} = \text{inode table start} + \text{inode offset}$

$12\text{KB} + 8192\text{bytes} = 12\text{KB} + 8\text{KB} = 20\text{KB}$

Step 4: Convert byte address to sector number

$\text{sector} = \text{absolute address} / \text{sector size} = 20,480 / 512 = 40$

Data Block Pointers in Inodes

- Inodes contain **direct pointers** to data blocks
- Each pointer references one disk block of the file
- **Limitation:**
 - File size limited by $\text{block size} \times \text{number of direct pointers}$
 - Large files need advanced schemes like **indirect pointers (as studied earlier)**

Directory Organization

- A directory is stored as a **list of entries** in its data blocks.
- Each entry represents a file or subdirectory.
- Each entry has:
- **inum**: the **inode number** of the file/directory
- **reclen**: **record length** (total size of this entry in bytes, including padding)
- **strlen**: **length of the name string**
- **name**: the **name** of the file or subdirectory

Example

Assume a directory `dir` (inode #5) contains three files: `foo`, `bar`, and `foobar_is_a_pretty_longname`, with inode numbers **12**, **13**, and **24**.

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname

- Each entry contains:
 - `inum`: inode number
 - `reclen`: total size of the entry (including padding)
 - `strlen`: actual length of the name
 - `name`: file or subdirectory name
- Directories always include:
 - `.` → current directory (inode #5)
 - `..` → parent directory (inode #2, the root)

These entries are stored in the directory's **data blocks** and enable directory navigation.

Handling File Deletion in Directories

- Deleting a file (e.g., via `unlink( )`) can leave **empty space** inside a directory's data block
- To mark deleted entries, a **reserved inode number** (such as 0) is used
- The **record length (recLEN)** allows the filesystem to:
 - **Reuse** the space of a deleted (larger) entry for new entries
 - Efficiently manage variable-sized directory entries without rewriting the whole directory

Next objectives

- Now that we understand how files and directories are stored on disk, let us explore how **reading and writing** actually happen
- Understanding the **access path** (steps taken by the file system) is crucial for grasping how file operations work
- Assumptions for examples:
 - File system is **mounted**
 - **Superblock** is loaded in memory
 - Other data (inodes, directories) remain on disk and must be accessed as needed

Scenario: Open and read file /foo/bar (12 KB, 3 blocks)

- **Open call:** `open("/foo/bar", O_RDONLY)`
 - FS needs **bar's inode** to get file info (size, permissions)
 - Only pathname given → FS must **traverse the path**
- **Path traversal:**
 - Start at root directory (inode #2, well-known)
 - Read root inode → get pointers to root's data blocks (directory entries)
 - Read directory blocks → find entry for foo and get its inode number (e.g., 44)
 - Read foo inode and data → find bar inode number

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
open(bar)			read		read	read		read		
read()					read			read		
read()					write					
read()					read				read	
read()					write					read

Scenario: Open and read file /foo/bar (12 KB, 3 blocks)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
open(bar)			read		read	read				
					read	read				
read()					read					
					write					
read()					read					
					write					
read()					read					
					write					
										read

- **Read bar's inode into memory:**

- Check permissions
- Allocate file descriptor for process

- **Reading file data:**

- read() consults bar's inode to find block locations
- Reads first block, updates file offset
- Subsequent reads access next blocks

- **Closing file:**

- Deallocate file descriptor
- No disk I/O needed

- **Notes:**

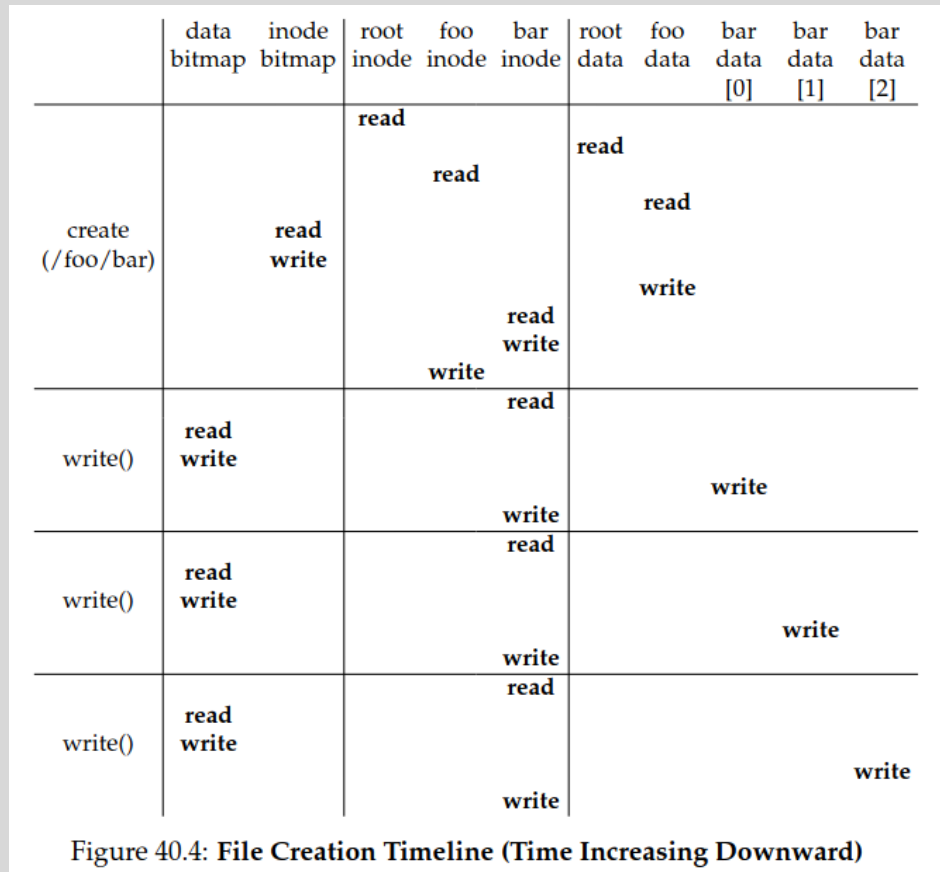
- I/O during open depends on **pathname length** and **directory size**
- Longer paths and large directories cause more disk reads

Writing A File To Disk

- When writing to a file like `/foo/bar`, the file system does a lot more than just writing the data itself.
- Before writing, the file needs to be created and opened.
- This involves locating the right place in the directory, allocating an inode (the file's metadata), and updating directory entries.

Writing Data Blocks

- Each time you write a block of data, the file system may need to allocate a new block on disk if it's writing new data (not overwriting).
- This allocation requires updating the **data bitmap** (which tracks free blocks) to mark the block as used.
- The inode must also be updated to include the new block's address.
- Because these metadata updates require disk reads and writes, each block write generates about 5 disk I/Os: reading and writing the inode, reading and writing the bitmap, and writing the data block itself.



Performance Concerns

- **Overall Cost:**

- This means even a small write involves many disk operations.
- The more data you write, the more expensive these metadata updates become.

- **Efficiency Concern:**

- With all these operations, how can file systems remain efficient?
- That's why they use optimizations like caching metadata in memory, batching writes, and pre-allocating blocks to reduce disk I/O.

Motivation Caching and Buffering

- Reading and writing files cause many slow disk I/Os
- Without caching, opening files with deep paths causes hundreds of reads
- Early file systems used fixed-size caches ($\sim 10\%$ of memory) with LRU to keep popular blocks
- Fixed vs. Dynamic Cache Partitioning
 - Fixed cache size wastes memory if not fully used
 - Modern OSs use dynamic partitioning via a unified page cache
 - Memory is shared flexibly between virtual memory and file system cache

Caching Effect on Reads

- First file open causes many disk reads (inodes + directories)
- Subsequent opens hit cache → no disk I/O needed

Caching Effect on Writes

- Writes must go to disk for persistence → cache can't filter write traffic fully
- Write buffering delays writes to:
 - Batch updates, reducing I/Os
 - Schedule writes for better performance
 - Avoid some writes entirely (e.g., create then delete a file)

Write Buffering

- Buffering writes in memory improves performance by:
 - Grouping multiple small writes into fewer large disk operations (batching)
 - Allowing the system to schedule disk writes efficiently
 - Avoiding some writes altogether (e.g., files created then deleted quickly)
- **Cost:**
 - If the system crashes before buffered writes reach the disk, those changes are lost, risking **data loss**.
- **Balance:**
 - Most systems delay writes for a short period (5–30 seconds) to optimize performance, accepting a small risk of data loss.
- **Special cases:**
 - Applications like databases that cannot tolerate data loss bypass buffering by forcing writes immediately (e.g., using `fsync( )` or direct I/O)



Thank You